

# Recuperacion de Bundle a Producto en Moda: Paper Tecnico y Documentacion del Sistema

Equipo Hack2026  
Informe Tecnico Interno

**Abstract**—Este documento describe el sistema completo de retrieval de Hack2026 para descomposicion de bundles de moda, donde cada imagen de bundle debe asociarse con uno o mas identificadores de producto. La implementacion combina un pipeline baseline de retrieval visual preentrenado y un pipeline avanzado de entrenamiento con OpenCLIP, con deteccion opcional de regiones de ropa. Se detallan el preprocesamiento de datos, el aumento offline, el entrenamiento del modelo, la validacion local, la inferencia y la generacion de entregas. El paper se apoya en el codigo del repositorio, con enfasis en reproducibilidad y decisiones de ingenieria.

**Index Terms**—recuperacion visual, aprendizaje multimodal, OpenCLIP, IA en moda, metric learning, matching a nivel de objeto

## I. INTRODUCCION

La tarea Hack2026 es un problema de retrieval visual multietiqueta: dada una imagen de bundle (un look o conjunto de prendas), el sistema debe predecir la lista de identificadores de producto (`product_asset_id`) presentes en ese bundle. La salida final es un CSV con una fila por producto predicho, agrupado por `bundle_asset_id`, con un maximo de 15 predicciones por bundle.

Este repositorio implementa dos enfoques complementarios:

- 1) Un baseline preentrenado ligero (`src/infer.py`) basado en retrieval por coseno sobre embeddings de imagen extraidos con backbones de torchvision.
- 2) Un modelo de retrieval multimodal entrenable (`src/models/retrieval_openclip.py`) basado en Marqo Fashion SigLIP, con deteccion opcional de regiones en bundles y optimizacion contrastiva.

Desde una perspectiva de ingenieria, el codigo se organiza en tres etapas:

- 1) Curacion de datos y preparacion de assets (`preprocess_data.py`, `download.sh`, `offline_augment.py`).
- 2) Entrenamiento y validacion local (`src/train.py` con configuracion Hydra).
- 3) Inferencia en test y exportacion de la entrega (`src/infer.py`, `src/infer_openclip.py`).

Este paper documenta el pipeline de extremo a extremo y las decisiones practicas incorporadas en la implementacion actual.

## II. DATOS Y PREPROCESAMIENTO

### A. Datasets de Origen

Las entradas principales son cuatro archivos CSV. Su rol funcional en el proyecto se resume en la Tabla I.

| Archivo                                              | Proposito                                                                     |
|------------------------------------------------------|-------------------------------------------------------------------------------|
| <code>data/bundles_dataset.csv</code>                | Catalogo de bundles y URLs de imagen de bundles.                              |
| <code>data/product_dataset.csv</code>                | Catalogo de productos, URLs de imagen de productos y descripciones textuales. |
| <code>data/bundles_product_mappings_train.csv</code> | Entrenamiento supervisado bundle-producto para entrenamiento.                 |
| <code>data/bundles_product_mappings_test.csv</code>  | Entrenamiento test donde se debe predecir <code>product_asset_id</code> .     |

TABLE I  
DATASETS DEL REPOSITORIO USADOS POR ENTRENAMIENTO E INFERENCIA.

### B. Pipeline de Preprocesamiento

El script `preprocess_data.py` ofrece:

- 1) Validaciones de esquema para columnas requeridas e integridad referencial.
- 2) Limpieza de etiquetas de descripcion de producto, incluyendo exclusion de categorias predefinidas de cosmetica/perfumeria.
- 3) Particion estratificada train/validacion por numero de etiquetas por bundle.
- 4) Descarga concurrente de imagenes con logica de reintentos y registro de URLs prohibidas.
- 5) Generacion de manifiestos JSONL y estadisticas de preprocesamiento.

El script genera:

- `manifests/train_manifest.jsonl`
- `manifests/val_manifest.jsonl`
- `labels.json`, `label2idx.json` y `stats.json`

### C. Aumento Offline

El script `offline_augment.py` genera vistas aumentadas deterministas, controladas por semilla, tanto para bundles como para productos. Aplica recortes redimensionados, jitter de color, flips horizontales opcionales, desenfoque gaussiano, cutout opcional (bundles) y perturbacion de calidad JPEG. Las imagenes aumentadas y los manifiestos se guardan como:

- `bundles_aug/*.jpg, bundles_aug_manifest.jsonl`
- `products_aug/*.jpg, products_aug_manifest.jsonl`

Este diseno esta alineado con la restriccion de una sola imagen por producto, donde las representaciones robustas de producto requieren expansion sintetica de vistas.

### III. METODOLOGIA

#### A. Formulacion del Problema

Para cada consulta de bundle  $b$ , el objetivo es ordenar candidatos de producto  $\{p_i\}_{i=1}^N$  y devolver hasta 15 IDs de producto unicos con mayor relevancia. El entrenamiento usa enlaces positivos conocidos de `bundles_product_match_train.csv`.

#### B. Retrieval Baseline (`src/infer.py`)

El baseline usa backbones preentrenados de torchvision (`resnet18`, `resnet50`, `efficientnet_b0`) para codificar imagenes de bundles y productos. Los embeddings se normalizan con L2 y la similitud coseno se calcula mediante producto punto.

Para cada embedding de consulta  $\mathbf{q}$  y matriz de productos  $\mathbf{P}$ :

$$\text{sim}(\mathbf{q}, \mathbf{P}) = \mathbf{q}\mathbf{P}^\top \quad (1)$$

El retrieval Top- $K$  se implementa con ordenacion parcial (`np.argsort`) por eficiencia.

#### C. Retrieval Multimodal con OpenCLIP

El backend avanzado usa Marqo Fashion SigLIP [1], [2]. Los embeddings de producto pueden fusionar representaciones de imagen y texto (tokens de descripcion de producto), mientras que las consultas de bundle son basadas en imagen.

El entrenamiento usa perdida contrastiva bidireccional simetrica:

$$\mathcal{L} = \frac{1}{2} \left[ \text{CE}((\mathbf{Q}\mathbf{P}^\top)/\tau, \mathbf{y}) + \text{CE}((\mathbf{P}\mathbf{Q}^\top)/\tau, \mathbf{y}) \right] \quad (2)$$

donde  $\tau$  es la temperatura y  $\mathbf{y} = [0, \dots, B-1]$  son los indices positivos dentro del batch.

#### D. Codificacion de Bundle Guiada por Deteccion

Cuando se activa (`params.use_bundle_boxes=true`), las imagenes de bundle se procesan con deteccion de ropa YOLO [3]. Cada region detectada se codifica, y los embeddings de region se promedian por bundle:

$$\mathbf{e}_b = \text{norm} \left( \frac{1}{M_b} \sum_{m=1}^{M_b} \mathbf{e}_{b,m} \right) \quad (3)$$

Si no se detectan cajas, se usa un fallback con la imagen completa. Los resultados de deteccion se guardan en cache en JSON para reutilizacion entre ejecuciones.

#### E. Metricas de Validacion

La validacion sigue un esquema de retrieval multitiqueta con Hit@K y Recall@K (implementado en `src/utis/metrics.py`). Dado el conjunto real  $G_b$  y las predicciones top- $K$   $\hat{G}_b^K$ :

$$\text{Recall@K}(b) = \frac{|G_b \cap \hat{G}_b^K|}{|\hat{G}_b^K|} \quad (4)$$

La metrica reportada es la media sobre los bundles evaluados.

### IV. DETALLES DE IMPLEMENTACION

#### A. Configuracion y Puntos de Entrada

El proyecto usa Hydra [4] con dataclasses tipadas (`src/config.py`). Los principales puntos de entrada son:

- `python -m src.train`: entrenamiento OpenCLIP.
- `python -m src.infer`: inferencia baseline preentrenada + particion de validacion local.
- `python -m src.infer_openclip`: inferencia desde un checkpoint entrenado de OpenCLIP.

Los parametros principales de ejecucion incluyen batch size, epocas, learning rate, AMP, acumulacion de gradiente, cortes de recall y ajustes opcionales de deteccion (`bbox_*`).

#### B. Arquitectura deCodigo

| Modulo                                        | Responsabilidad                                                          |
|-----------------------------------------------|--------------------------------------------------------------------------|
| <code>src/train.py</code>                     | Bootstrap de Hydra y despacho al trainer.                                |
| <code>src/models/retrieval_openclip.py</code> | Backend principal de entrenamiento/validacion OpenCLIP y checkpoints.    |
| <code>src/infer.py</code>                     | Extraccion de embeddings baseline, retrieval y exportacion de entrega.   |
| <code>src/infer_openclip.py</code>            | Inferencia OpenCLIP para bundles de test desde checkpoint.               |
| <code>src/detection.py</code>                 | Wrapper del detector YOLO y utilidades de extraccion de cajas.           |
| <code>src/utis/retrieval.py</code>            | Helpers de similitud y retrieval Top-K.                                  |
| <code>src/utis/metrics.py</code>              | Calculo de Hit@K y Recall@K.                                             |
| <code>preprocess_data.py</code>               | Validacion de integridad del dataset, split, manifiestos y estadisticas. |
| <code>offline_augment.py</code>               | Aumento offline y manifiestos de aumento.                                |

TABLE II

MODULOS PRINCIPALES Y RESPONSABILIDADES.

#### C. Aspectos Clave del Loop de Entrenamiento

La implementacion de entrenamiento OpenCLIP incluye:

- 1) Resolucion dinamica de dispositivo (CPU/CUDA) y DataParallel multi-GPU opcional.
- 2) Scheduler de learning rate coseno con 10% de warmup.
- 3) Precision mixta opcional via `torch.cuda.amp`.
- 4) Logging periodico, validacion por epoca, anexo de metricas en JSONL y checkpoints (`epoch_X.pt`, `best.pt`).

### D. Logica de Inferencia y Fallback

Ambos scripts de inferencia generan una fila por producto predicho y limitan la salida a 15 por bundle. Si no hay embeddings de bundle disponibles (por ejemplo, imagen ausente), se usan como fallback productos populares obtenidos de enlaces de train.

## V. PROTOCOLO EXPERIMENTAL

### A. Estrategia de Evaluacion Local

El repositorio ofrece dos rutas de validacion local:

- 1) Modo baseline (`src/infer.py`) muestrea bundles de validacion con `infer.val_ratio` y reporta `hit@K/recall@K`.
- 2) Modo `entrenamiento` (`src/models/retrieval_openclip.py`) evalua `recall@K` tras cada epoca, donde  $K = \text{params.recall\_k}$ .

Los productos se indexan una vez por pasada de validacion, los bundles se codifican y los vecinos top-K mas cercanos se recuperan por similitud coseno.

### B. Artefactos de Salida

Principales artefactos generados por el flujo de entrenamiento/inferencia:

- `metrics.jsonl`: perdida de entrenamiento por epoca y recall de validacion.
- `best.pt`: mejor checkpoint segun la metrica de recall.
- `test_submission.csv`: archivo final de predicciones para subir a la competicion.
- `val_metrics.json`: resumen de ejecucion baseline y metricas locales.

### C. Ablaciones Recomendadas

Para medir el impacto de los componentes principales, se recomiendan las siguientes ablaciones:

- Productos solo imagen vs. embeddings de producto imagen+texto.
- Codificacion de bundle con imagen completa vs. agregacion de regiones guiada por deteccion.
- Sin aumento offline vs. aumento offline determinista.
- Single-GPU vs. multi-GPU, con semilla fija y mismo batch efectivo.

## VI. REPRODUCIBILIDAD

### A. Entorno

Las dependencias base estan listadas en `requirements.txt`: `pandas`, `numpy`, `pillow`, `tqdm`, `requests`, `hydra-core`, `torch` y `torchvision`. Paquetes adicionales requeridos por funciones avanzadas:

- `open_clip_torch` para el backend OpenCLIP.
- `ultralyticsplus` para la deteccion de ropa en bundles.

### B. Controles de Determinismo

El codigo fija explicitamente semillas aleatorias (`random`, `numpy`, `torch`). El aumento offline usa semillas estables basadas en hash por asset e indice de aumento. Esto permite preprocesamiento repetible y comportamiento de entrenamiento consistente bajo configuracion fija.

### C. Configuracion Hydra

La configuracion se divide en:

- `config/config.yaml`: parametros de entrenamiento e inferencia.
- `config/files/file.yaml`: rutas de dataset/imagenes/cache.

Los directorios de salida de Hydra guardan artefactos especificos de cada ejecucion, haciendo los experimentos trazables sin gestion manual de rutas.

### D. Contrato de Entrega

El CSV final debe contener:

- `bundle_asset_id`
- `product_asset_id`

con hasta 15 filas por bundle, ordenadas por confianza del modelo.

## VII. LIMITACIONES Y RIESGOS

- 1) **Supervision de producto en vista unica:** cada ID de producto esta representado por imagenes muy limitadas, lo que puede reducir la robustez ante cambios de pose, oclusion e iluminacion.
- 2) **Manifiesto de validacion por defecto en el endpoint de entrenamiento:** actualmente `src/train.py` apunta tanto train como val a `bundles_product_match_train.csv`; para una seleccion estricta del modelo se deben pasar manifiestos de validacion separados.
- 3) **Dependencia de deteccion:** el modo guiado por deteccion depende de checkpoints externos de YOLO e introduce sobrecoste computacional adicional.
- 4) **Calibracion de la entrega:** la calidad del ranking importa mas que la exactitud top-1 simple porque la evaluacion conserva hasta las primeras 15 predicciones por bundle.

Las siguientes mejoras recomendadas son hard-negative mining, indexado ANN (por ejemplo, FAISS), estrategias de codificacion de texto mas ricas y capas de reranking mas potentes.

## VIII. CONCLUSIONES

El repositorio de Hack2026 ya contiene un stack completo de retrieval: preparacion de datos, aumento determinista, retrieval baseline, entrenamiento OpenCLIP, codificacion opcional con sensibilidad a regiones y generacion de entregas. El diseno actual es practico y orientado a produccion, con configuracion explicita, checkpoints y registro de metricas.

Este paper funciona como especificacion tecnica del sistema implementado y puede usarse como base para trabajo futuro en precision de retrieval, optimizacion de latencia y reranking de candidatos a gran escala.

## REFERENCES

- [1] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, “Sigmoid loss for language image pre-training,” *arXiv preprint arXiv:2303.15343*, 2023.
- [2] A. Radford, J. W. Kim, C. Hallacy *et al.*, “Learning transferable visual models from natural language supervision,” in *Proceedings of the International Conference on Machine Learning*, 2021.
- [3] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics yolov8,” GitHub repository, 2023.
- [4] O. Yadan, “Hydra: A framework for elegantly configuring complex applications,” GitHub repository, 2019.

## APPENDIX

### APENDICE: COMANDOS REPRODUCIBLES

#### A. Descarga de Datos

```
bash download.sh
```

#### B. Preprocesamiento

```
python preprocess_data.py \  
  --skip_download \  
  --out_dir data/preprocessed \  
  --val_ratio 0.1 \  
  --seed 42
```

#### C. Aumento Offline

```
python offline_augment.py \  
  --bundles_manifest data/manifests/train_manifest.jsonl \  
  --products_manifest data/product_dataset.csv \  
  --products_images_dir data/product_images \  
  --out_dir data/offline_aug \  
  --bundles_num_augs 4 \  
  --products_num_augs 2 \  
  --img_size 224 \  
  --seed 42 \  
  --workers 8
```

#### D. Entrenamiento (Hydra)

```
python -m src.train
```

#### E. Inferencia Baseline

```
python -m src.infer
```

#### F. Inferencia OpenCLIP

```
python -m src.infer_openclip \  
  --checkpoint outputs/retrieval_openclip/best.pt \  
  --submission-out outputs/retrieval_openclip/submission.csv
```